CSCI 4730/6730
(Operating Systems – PA2)

In Kee Kim

School of Computing

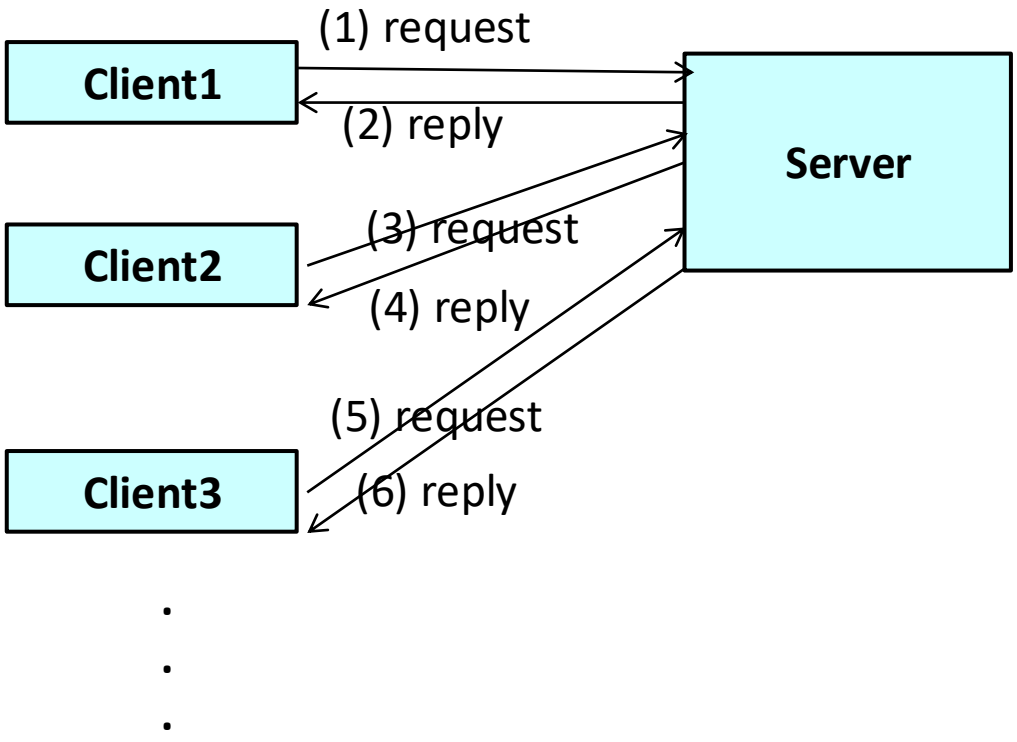
University of Georgia

PA #2 -- Goal

- Multi-threaded webserver with Synchronization
- Thread Pool
- **Consumer Producer Model**

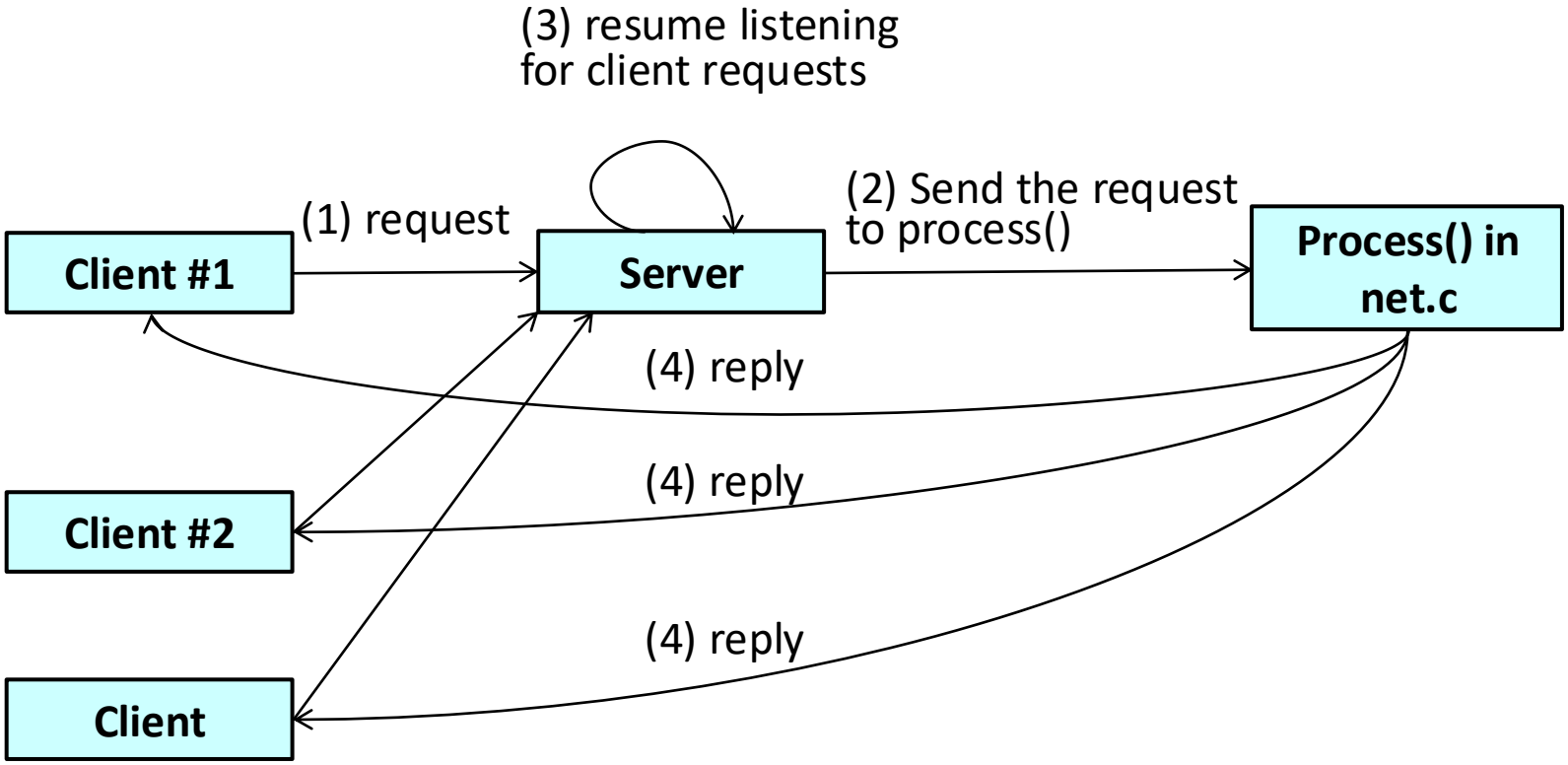
Base Code: `webserver.c`

- A single thread-version of web server



Base Code: webserver.c

- A single thread-version of web server



Base Code: webserver.c

```
 1: #include <stdio.h>
 2: #include <string.h>
 3: #include <time.h>
 4: #include <sys/socket.h>
 5: #include <linux/unistd.h>
 6: #include <arpa/inet.h>
 7: #include "webserver.h"
 8:
 9: int listener(int port)
10: {
11:   int sock;
12:   struct sockaddr_in sin;
13:   struct sockaddr_in peer;
14:   int peer_len = sizeof(peer);
15:   int r;
16:
17:   sock = socket(AF_INET, SOCK_STREAM, 0);
18:   sin.sin_family = AF_INET;
19:   sin.sin_addr.s_addr = INADDR_ANY;
20:   sin.sin_port = htons(port);
21:   r = bind(sock, (struct sockaddr *) &sin, sizeof(sin));
22:   if(r < 0) {
23:     perror("Error binding socket:");
24:     return -1;
25:   }
26:
27:   r = listen(sock, 5);
28:   if(r < 0) {
29:     perror("Error listening socket:");
30:     return -1;
31:   }
32:
33:   printf("HTTP server listening on port %d\n", port);
34:   while (1) {
35:     int fd;
36:     fd = accept(sock, NULL, NULL);
37:     if (fd < 0) {
38:       printf("Accept failed.\n");
39:       break;
40:     }
41:     process(fd);
42:   }
43:   close(sock);
44:
45: }
46:
```

```
47: int main(int argc, char *argv[]) {
48:   if(argc != 2 || atoi(argv[1]) < 2000 || atoi(argv[1]) > 50000)
49:   {
50:     fprintf(stderr, "./webserver PORT(2001 ~ 49999)\n");
51:     return 0;
52:   }
53:
54:   int port = atoi(argv[1]);
55:   listener(port);
56:   return 0;
57: }
```

```
./webserver 5001
```

How to run webserver?

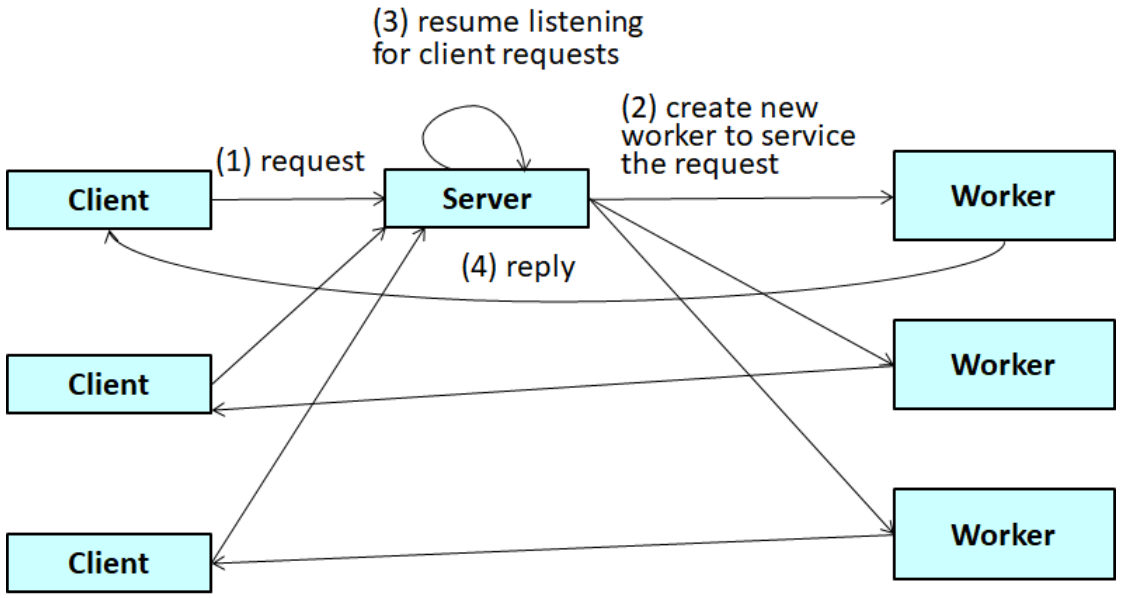
- Two terminals
  - One terminal for server:
    - e.g., **`./webserver 5001`**
  - The other terminal for client:
    - e.g., **`./client 127.0.0.1 5001 10`**

`process()` in `net.c`

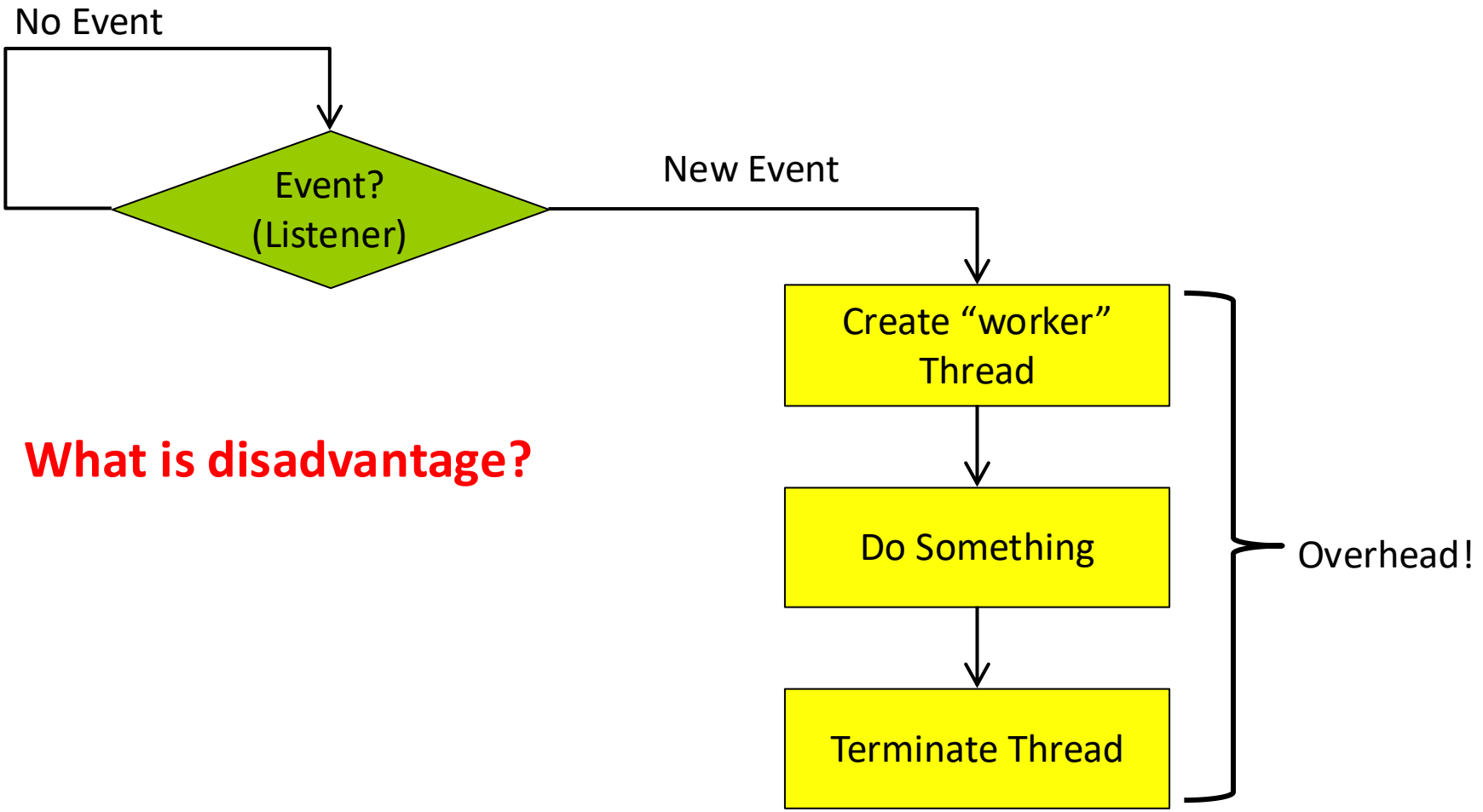
- You don't have to change this function.
- **`webserver.c`** is even worse than typical web serve architecture

Thread Model in Web Server

- Typical Thread Model in a “web server”
  - Web server handles multiple client connections simultaneously



Typical Thread Model in “Server”

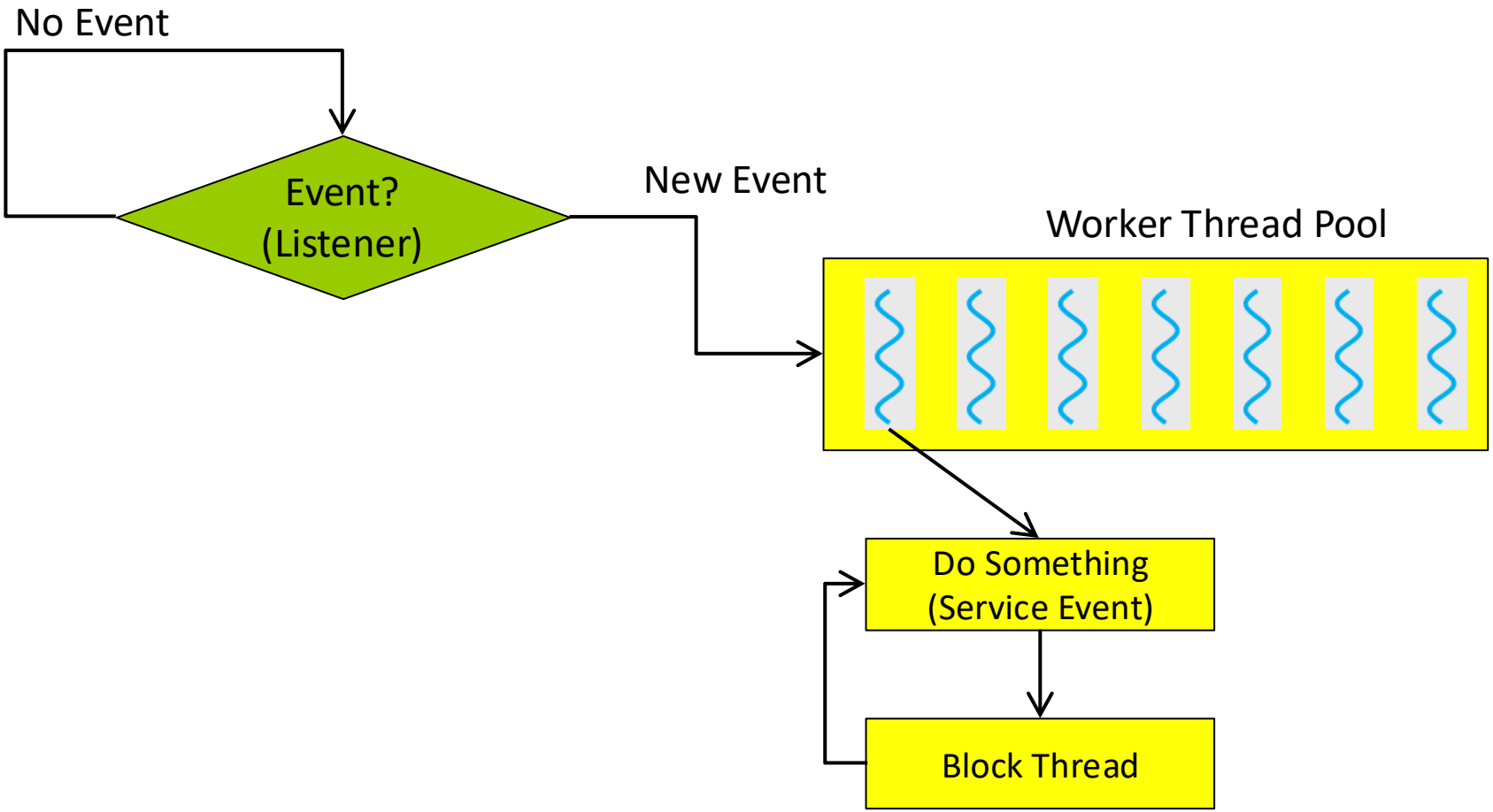


What is disadvantage?

Thread Pools

- You need to improve webserver with thread pool

Thread Pools



Thread Pools

- Create a number of threads in a pool where they await work
- What are Advantages:
  - Usually ***slightly*** faster to service a request with an existing thread than create a new thread
    - No Thread Startup/Creation Delay
  - Allows the number of threads in the application(s) to be bound to the size of the pool

webserver_multi.c: template()

```
 1: #include <stdio.h>
 2: #include <stdlib.h>
 3: #include <arpa/inet.h>
 4: #include <pthread.h>
 5: #include "webserver.h"
 6:
 7: #define MAX_REQUEST 100
 8:
 9: int port, numThread;
10:
11: void *listener()
12: {
13:   int r;
14:   struct sockaddr_in sin;
15:   struct sockaddr_in peer;
16:   int peer_len = sizeof(peer);
17:   int sock;
18:
19:   sock = socket(AF_INET, SOCK_STREAM, 0);
20:   sin.sin_family = AF_INET;
21:   sin.sin_addr.s_addr = INADDR_ANY;
22:   sin.sin_port = htons(port);
23:   r = bind(sock, (struct sockaddr *) &sin, sizeof(sin));
24:   if(r < 0) {
25:     perror("Error binding socket:");
26:     return;
27:   }
28:
29:   r = listen(sock, 5);
30:   if(r < 0) {
31:     perror("Error listening socket:");
32:     return;
33:   }
34:
35:   printf("HTTP server listening on port %d\n", port);
36:   while (1)
37:   {
38:     int s;
39:     s = accept(sock, NULL, NULL);
40:     if (s < 0) break;
41:
42:     //process(s);
43:   }
44:   close(sock);
45: }
```

```
47: void thread_control()
48: {
49:   /* ----- */
50: }
51:
52: int main(int argc, char *argv[])
53: {
54:   if(argc != 3 || atoi(argv[1]) < 2000 || atoi(argv[1]) > 50000)
55:   {
56:     fprintf(stderr, "./webserver_multi PORT(2001 ~ 49999) #_of_threads\n");
57:     return 0;
58:   }
59:
60:   int i;
61:   port = atoi(argv[1]);
62:   numThread = atoi(argv[2]);
63:   thread_control();
64:   return 0;
65: }
```

```
./webserver_multi 5001 10
```

of worker threads

main() difference

webserver.c

```
47: int main(int argc, char *argv[]) {
48:   if(argc != 2 || atoi(argv[1]) < 2000 || atoi(argv[1]) > 50000)
49:   {
50:     fprintf(stderr, "./webserver PORT(2001 ~ 49999)\n");
51:     return 0;
52:   }
53:
54:   int port = atoi(argv[1]);
55:   listener(port);
56:   return 0;
57: }
```

Webserver_multi.c

sem, mutex init ------>>

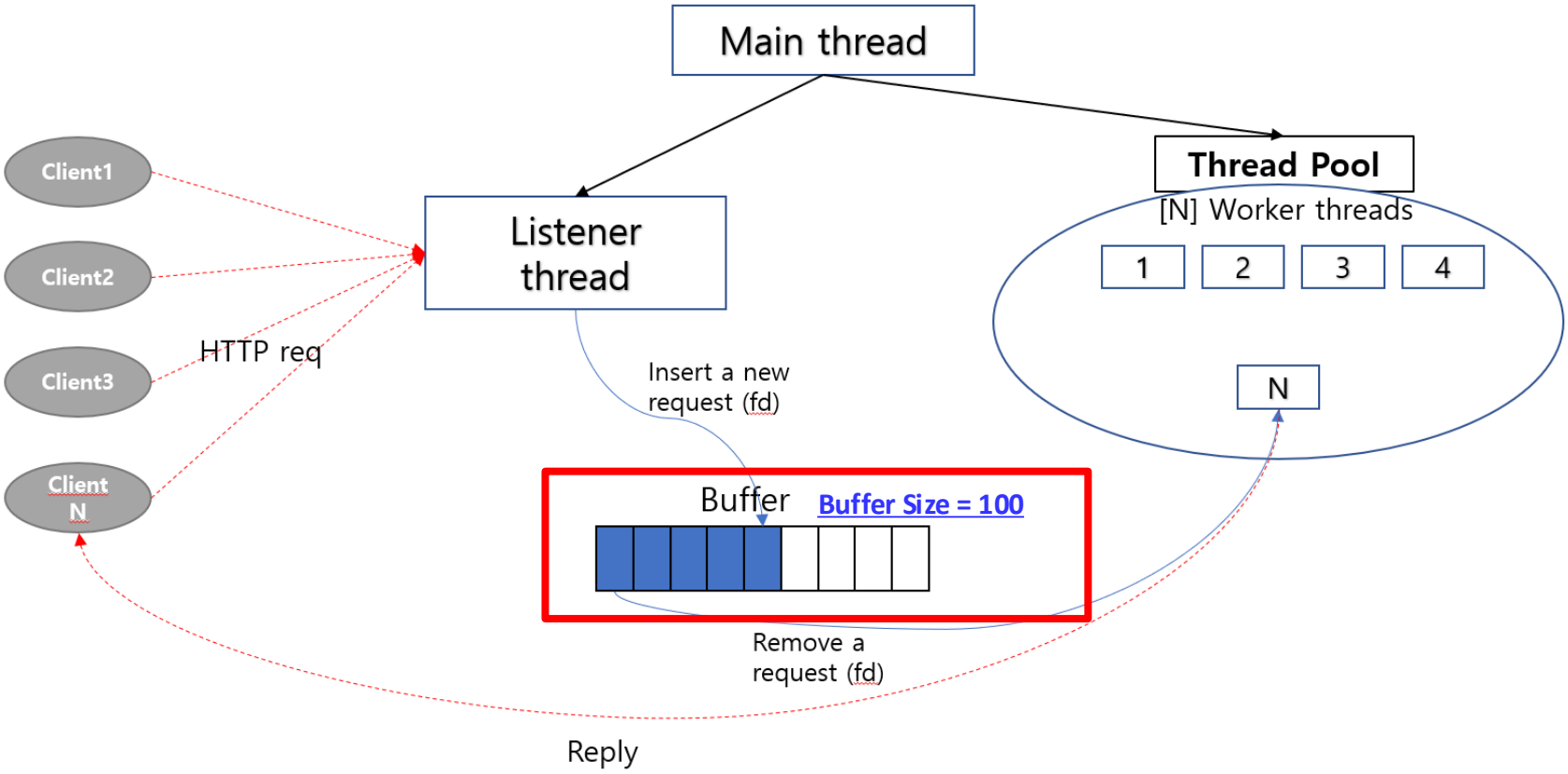
```
47: void thread_control()
48: {
49:   /* ----- */
50: }
51:
52: int main(int argc, char *argv[])
53: {
54:   if(argc != 3 || atoi(argv[1]) < 2000 || atoi(argv[1]) > 50000)
55:   {
56:     fprintf(stderr, "./webserver_multi PORT(2001 ~ 49999) #_of_threads\n");
57:     return 0;
58:   }
59:
60:   int i;
61:   port = atoi(argv[1]);
62:   numThread = atoi(argv[2]);
63:   thread_control();
64:   return 0;
65: }
```

<<--- sem, mutex destroy before return

What to add in `thread_control()`

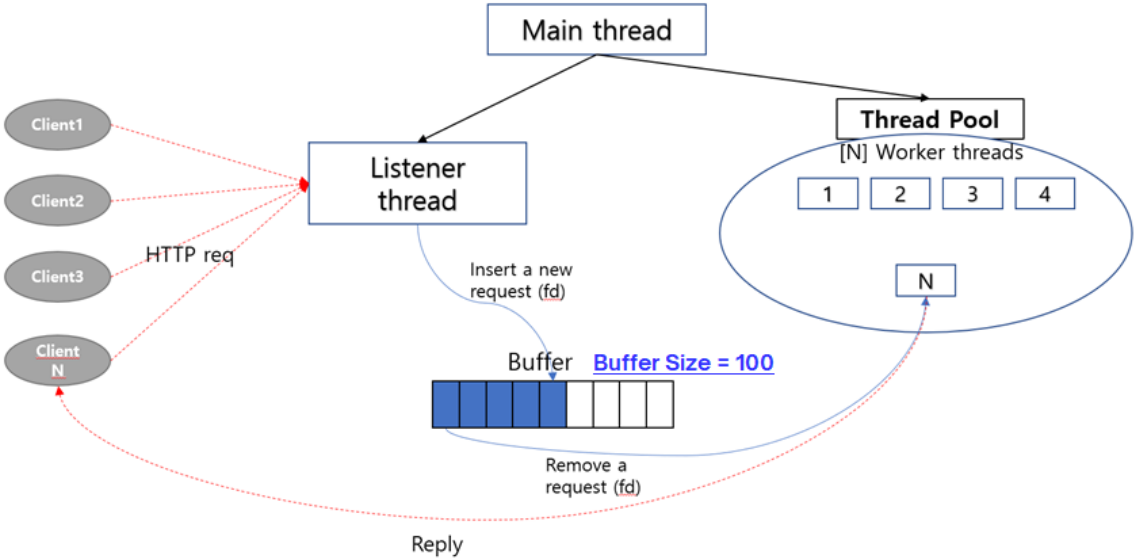
- Lines 48 – 51 in webserver_multi.c template
- Create thread pool that has ***N+1*** threads
  - 1 ***listener*** thread (pthread_t)
  - N ***worker*** threads (pthread_t)
    - N: # of thread argument
    - **`./webserver_multi 5001 10`**
    - Max number of worker threads: 100
    - You can use either static or dynamic approach (or array or pointer)
- pthread_join() for the ***N+1*** threads

Overall Workflow



This is typical producer-consumer model

- You need to create 1 **producer** and N **consumers**
  - Producer
    - Producer is part of listener
    - Producer inserts http requests to buffer



This is typical producer-consumer model

- You need to create 1 **producer** and N **consumers**
  - Producer
    - Producer is part of listener
    - Producer inserts http requests to buffer

```
 1: #include <stdio.h>
 2: #include <stdlib.h>
 3: #include <arpa/inet.h>
 4: #include <pthread.h>
 5: #include "webserver.h"
 6:
 7: #define MAX_REQUEST 100
 8:
 9: int port, numThread;
10:
11: void *listener()
12: {
13:   int r;
14:   struct sockaddr_in sin;
15:   struct sockaddr_in peer;
16:   int peer_len = sizeof(peer);
17:   int sock;
18:
19:   sock = socket(AF_INET, SOCK_STREAM, 0);
20:   sin.sin_family = AF_INET;
21:   sin.sin_addr.s_addr = INADDR_ANY;
22:   sin.sin_port = htons(port);
23:   r = bind(sock, (struct sockaddr *) &sin, sizeof(sin));
24:   if(r < 0) {
25:     perror("Error binding socket:");
26:     return;
27:   }
28:
29:   r = listen(sock, 5);
30:   if(r < 0) {
31:     perror("Error listening socket:");
32:     return;
33:   }
34:
35:   printf("HTTP server listening on port %d\n", port);
36:   while (1)
37:   {
38:     int s;
39:     s = accept(sock, NULL, NULL);
40:     if (s < 0) break;
41:
42:     //process(s);   producer(s)
43:   }
44:   close(sock);
45: }
46:
```

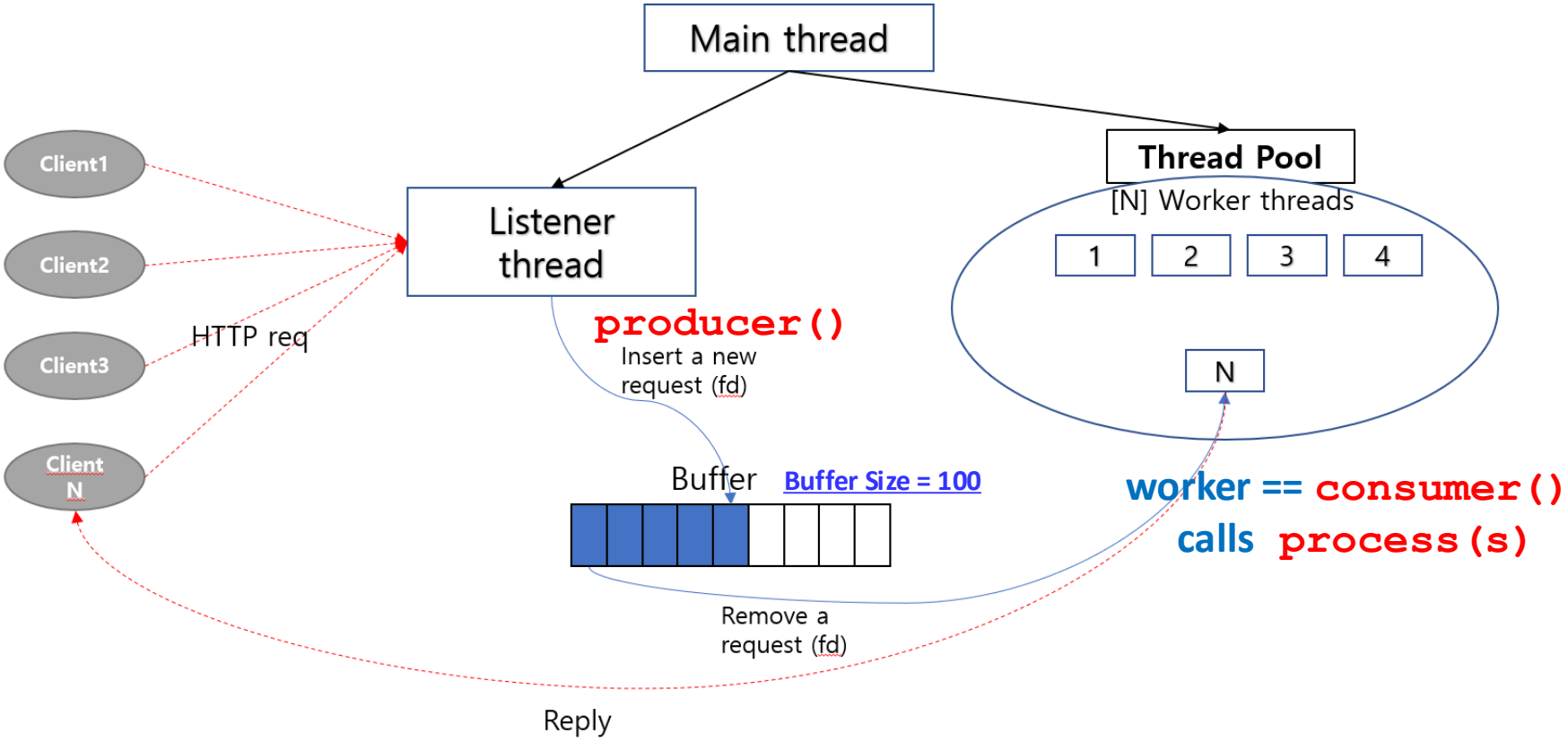
This is typical producer-consumer model

- You need to create 1 **producer** and N **consumers**
  - *N* Consumers
    - N worker threads
    - Reads HTTP request from buffer
    - Call `process(s);`

This is typical producer-consumer model

- You need to create 1 **producer** and N **consumers**
  - Requirement
    - Two semaphores; **`sem_empty`, `sem_full`**
      - **`sem_empty`:** # of empty buffers, init: MAX_REQUEST
      - **`sem_full`:** # of full buffers, init: 0
    - One Mutex lock; **`mutex`**

This is typical producer-consumer model



Submission

- Submit a tarball file using the following command
  - `%tar czvf pro2.tar.gz README.pdf Makefile webserver_multi.c`
- README.pdf file with:
  - Your name
  - Explain your design of shared data structure and synchronization (and show that your solution does not have any concurrency problems).
  - Also, you need to explain how you tested and verified your program against race conditions and deadlocks.
- All source files needed to compile, run and test your code
  - Makefile, All source files, Do not submit object or executable files
- Your code should be compiled and run correctly in odin machine.
  - **No credit will be given if your code failed to compile with "make" in odin.cs.uga.edu** (we will use your makefile).
- **Submit a tarball through ELC.**

Note

- When test **`webserver`** or **`webserver_multi`**, you may need to change port number frequently
- To get full credit, you should correctly use semaphore, mutex lock, and consumer-producer model.
- The maximum number of threads in the webserver is 100.
- With a larger number of threads, the webserver should show the performance improvement
  - Shorter end-to-end request processing time
- With a smaller number of threads, the webserver should show the performance degradation.
  - Longer end-to-end request processing time

Note

- Deadline
  - 11:59 pm, 10/26/2026 (Saturday)
- Recommendation
  - Nice to finish this before midterm as BB is part of the exam
- The late submission policy will be applied.